SmartCare Case Study

Current System:

The Current System Manages Patient Information Through:
- Spreadsheets
- Paper Records
- Manual Processes

Problems Outlined By Client:
- Duplicate Appointment Bookings
- Difficulty Locating Patient Records
- Inconsistent Appointment Status Information
- Limited Visibility Of Practitioner Availability
- Manual Cancellation Processes (Physical, Phone)
- Lack Of Reliable Appointment History.
- Difficulty Producing Basic Operational Reports.

Client Wants:
- Simple Software System

Client Wants, Personal Assumption Based Off Problems:

User/Patient System Assumptions:
- Client Requires A System That Allows Them To Book Appointments.

- The System Should Recognise When An Appointment Is Already Booked.

- The System Should Allow Users To Manage Appointments.

- The System Should Allow Users To View The Availability Of Practitioners.

Staff/Practitioner Assumptions:
- The System Should Have A Centralised Database Of Patient Records, Which Also Has A Search Function. This Database Should Also Be Connected To Another Database That Has Appointment History.

- The System Should Be Able To Produce Basic Operational Reports.

SmartCare Case Study

Activity 1:

If ChatGPT or Copilot can produce a 100-line Python application very quickly, what knowledge does a software Engineer still need?

Generative AI may be able to generate code for applications, however what they lack is context. To further elaborate they are lacking in the design phase where planning and analysis is required. Artificial Intelligence may be able to follow instructions, and wants provided by the prompt yet the method in which they achieve the solution is rigid or incorrect. Lastly Generative AI cannot produce what they aren't prompted.

Example:
A Baker prompts Generative Artificial Intelligence to program an interactive website for their business. The Ai in this case would most likely return with the frontend of the website, the front end may entail the correct menu items, opening hours, and even stylistic features yet there will be no backend. Yet the Backend is extremely important to any functioning website found today, the backend is what would allow the baker to change prices, to add new menu items and to change opening times. Software Engineers are needed because they have knowledge and context.

Activity 2 - Is This Software Engineering:

Scenario A: A student writes a 50-line Python calculator.
- Scenario A is considered Programming as the student wrote 50 lines of code in Python to solve a specific problem. Software Engineering is an elaborate process where requirement analysis, system design, testing, documentation and maintainability are incorporated.

Scenario B: A team develops a payroll system used by 5,000 employees.
- Scenario B is considered Software Engineering as an entire team "develops" a payroll system. Development would entail following all of the processes listed in my previous elaboration.

Scenario C: An AI assistant generates a simple appointment application from one prompt.
- Scenario C is considered Programming, As stated in Activity 1, Generative Ai lacks contextual capabilities. This is further proved as the scenario states "one prompt", therefore the Software Engineering Process was likely never considered in the first place.

SmartCare Case Study

Activity 3 - SmartCare Problem Analysis:

Task 1 - Identify Stakeholders:

- Patients
- Management
- Healthcare Practitioners
- Organisation

Patients - What Do They Need?:
- A System That Allows Patients To Book Appointments.
- A System That Allows Patients To Manage Appointments.
- A System That Allows Patients To View The Availability Of Practitioners.

Management - What Do They Need?:
- A System Which Recognises Duplicate Appointments And Can Cancel Them.

- A System That Can Produce Basic Operational Reports.

- A Simple Software System Which Allows Initial Support Of Patients, Practitioners, And Appointment Management.

Healthcare Practitioners - What Do They Need?:
- A System Which Allows Them To Access And Search For Patient Records.

- A System Which Allows Them To Access And Search For Appointment Histories.

Organisation - What Do They Need?:
- A System That Is Not Complex, The System Should Not Be Comparable To Hospital Information Systems.

- The First Iteration Of The System Should Be A Manageable Application, Suited For A Small Clinic.

SmartCare Case Study

Activity 3 - SmartCare Problem Analysis:

Task 2 - Identify Current Problems:

1. Duplicate Appointment Bookings.

2. Difficulty Locating Patient Records.

3. Important Information Scattered In Spreadsheets, Paper Records, And Manual Processes.

4. Difficulty Producing Operational Reports.

Task 3 - Ask Client Questions:

1. I Would Like Further Elaboration On The Producing Of Operational Reports.

2. Elaborate On The Term Simple System, Which Parts Would You Prefer Simplicity To Be Incorporated? Perhaps A Simple Frontend?

3. Could You Elaborate On How You Would Like The Issue On Patient Records Solved?

4. In Appointment Management How Would You Like To See Appointment Rescheduling Implemented?

5. Could You Elaborate Further On Appointment Status Information?

SmartCare Case Study

Activity 4 - Critique an AI Response:

Appointment Management:

- Client Evidence: Duplicate Appointment Bookings, Manual Cancellation Processes, Inconsistent Appointment Status Information.

- In Scope? Yes

- Decision: This Is One Of The Systems The Client Required.

Facial Recognition Login:

- Client Evidence: Simple Software System

- In Scope? No, this system is not simple in the slightest. Furthermore, cybersecurity needs to be heavily considered when facial recognition systems are concerned. The case study also states the Individual designing the system Is a junior software engineer, which would lack the capability and experience to achieve so In the first place.

- Decision: No.

AI Diagnosis Recommendations:

- Client Evidence: "SmartCare is a small community clinic that provides consultations through a number of healthcare practitioners"

- In Scope? No, this feature shares issues with the last suggestion. Furthermore ethicality needs to be considered as well.

- Decision: No.

Patient Search:

- Client Evidence: Difficulty Locating Patient Records, Lack Of Reliable Appointment History.

- In Scope? Yes, This system would require knowledge and experience with databases which a junior software developer definitely should have.

- Decision: Yes, This Is One Of The Systems The Client Required.

SmartCare Case Study

Activity 4 - Critique an AI Response:

Online Payment:

- Client Evidence: “Management wants a simple software system that can initially support patient, practitioner and appointment management.”

- In Scope? Yes, Online Payment systems are often not complex as companies like Visa have standardised online payment systems.

- Decision: No, This feature opposes one of the clients key requirements. Furthermore this feature may conflict with other systems causing issues.

Practitioner Schedule View:

- Client Evidence: Limited Visibility Of Practitioner Availability

- In Scope? Yes, This feature would not require extensive experience nor expertise. Having the work schedules of practitioners and knowledge on functionality of databases would allow this to be implemented.

- Decision: Yes, This Is One Of The Systems The Client Required.

Insurance Processing:

- Client Evidence: “Management wants a simple software system that can initially support patient, practitioner and appointment management.”

- In Scope? No, This feature would require incorporating multiple systems together simultaneously.

- Decision: No, This feature opposes one of the clients key requirements. Furthermore this feature may conflict with other systems causing issues.

Treatment-Plan Generation:

- Client Evidence: “Management wants a simple software system that can initially support patient, practitioner and appointment management.”

- In Scope? No, shares the same issues with Ai Diagnosis Recommendations.

- Decision: No, This feature opposes one of the clients key requirements. Furthermore this feature places major ethical risks on the clinic.

SmartCare Case Study

Exit Question:

Write one activity that a software engineer must perform and that cannot safely be delegated entirely to AI.

A Program that must never be delegated to Artificial Intelligence Is the training and management of Artificial Intelligence Itself.

Initial Engineering Brief

1. Problem summary

SmartCare is a small community clinic, which currently struggles with a range of problems. The Management side of their problems arise from the fact that the majority of their important information is scattered in different forms such as; Spreadsheets, Physical Records And Manual Processes. The clinic's other problems are related to patient accessibility ranging from appointment management to appointment related information.

2. Initial Stakeholders:

- Patients
- Management
- Healthcare Practitioners
- Organisation

Patients - What Do They Need?:
- A System That Allows Patients To Book Appointments.
- A System That Allows Patients To Manage Appointments.
- A System That Allows Patients To View The Availability Of Practitioners.

Management - What Do They Need?:
- A System Which Recognises Duplicate Appointments And Can Cancel Them.

- A System That Can Produce Basic Operational Reports.

- A Simple Software System Which Allows Initial Support Of Patients, Practitioners, And Appointment Management.

Healthcare Practitioners - What Do They Need?:
- A System Which Allows Them To Access And Search For Patient Records.

- A System Which Allows Them To Access And Search For Appointment Histories.

Organisation - What Do They Need?:
- A System That Is Not Complex, The System Should Not Be Comparable To Hospital Information Systems.

- The First Iteration Of The System Should Be A Manageable Application, Suited For A Small Clinic.

Initial Engineering Brief

3. Initial features:

1. Appointment Management:
The Appointment Management Feature Will Allow Authorised Users To Book, Cancel And Reschedule Appointments. This Feature Will Also Allow The Authorised User To Select A Preferred Practitioner, And View Practitioner Availability. Lastly This Feature Would Also State The Current Status Of Appointments Such As: Scheduled, Cancelled, Completed, Rescheduled.

1.1. This feature is provisional and subject to change. I have chosen this feature as it addresses several of SmartCare's current problems: Duplicate Booking, Limited Visibility Of Practitioner Availability, Inconsistent Appointment Status Information, And Manual Cancellation Process Issues.

2. Patient Management:
The Patient Management Feature Will Allow Authorised Users To Browse, View, And Edit Patient Records. Furthermore The Existence Of This Feature Would Centralise All Patient Records, This Would Mean There Would No Longer Be Issues With Locating Patient Records.

2.1. This feature is provisional and subject to change and incorporation into other features. I have chosen this feature as it addresses SmartCare's problem relating to difficulty locating patient records. This feature would centralise all patient records in a database, allowing for a consistent method in obtaining aforementioned records.

3. Appointment History:
The Appointment History Feature Will Allow Authorised Management And Health Care Practitioners To View Relevant Historical Appointment Information. This Feature Will Provide A Reliable Method To Obtain Appointment History, Such As: Dates, Practitioners Visited, Appointment Statues, Medical History, Medication History ect.

3.1. This feature is provisional and subject to change and incorporation into other features. I have chosen this feature as it addresses SmartCare's lack of ability to reliably obtain appointment history.

Initial Engineering Brief

3. Initial features:

4. Operational Reporting:
The Operational Reporting Feature Will Allow Authorised Management To View, And Produce Basic Operational Reports. This Feature Would Achieve This By Collecting Information From The Other System Features. The Reports In This Case Could Provide Information Retaining To Appointments, Practitioner Availability, Appointment Activity, Cancellations, Reschedules etc.

4.1. This feature is provisional and subject to change. I have chosen this feature as it addresses SmartCare's difficulty in producing basic operational reports which I assume originates from their current system which is extremely scattered in relation to where important information is located.

4. Questions For The Client:

1. I Would Like Further Elaboration On The Producing Of Operational Reports.

2. Elaborate On The Term Simple System, Which Parts Would You Prefer Simplicity To Be Incorporated? Perhaps A Simple Frontend?

3. Could You Elaborate On How You Would Like The Issue On Patient Records Solved?

4. In Appointment Management How Would You Like To See Appointment Rescheduling Implemented?

5. Could You Elaborate Further On Appointment Status Information?

5. What We Do Not Yet Know?

1. The Types Of Programming Language We Are Allowed To Use In The Final System.

2. What Is Considered Complex To The Client? Many Small Clinics Have Systems With Backends That Cannot Be Considered Simple Yet This Allows The Frontend To Be Extremely Simple And Accessible To Both Patients, Management, And Practitioners.

Initial Engineering Brief

Part A - Understand the Problem:

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

- Patient Name
- Practitioner Name
- Appointment Time
- Appointment Date

What functions might be useful?

- Add New Appointment
- View Existing Appointments
- Search Function For Appointments, And Practitioners.
- Appointment Edit Function
- Delete/Cancel Appointment
- Schedule Conflict Warning/Indicator
- Validation That Required Information Has Been Provided Or Entered Correctly.

What could go wrong?

- The receptionist enters incorrect patient information
- A Required Field Is Left Blank
- Invalid Date Or Time Is Entered, Can Be Format As Well.
- Two Appointments Are Scheduled At The Same Time.
- Data Is Not Formatted Properly And Data Becomes Inconsistent
- Two Patients Could Have The Same Name.

Initial Engineering Brief

What requirements are unclear?
- What Types Of Appointments Are There? Most Modern Day Clinics Have Booking Systems For Both Short Form And Long Form Appointments.

- Can Appointments Be Cancelled Or Rescheduled?

- Are There Multiple Receptionists Using The System, If So Could Conflicts Be A Problem?

- Does The System Require Login/Security, As Patient Information Is Involved?

- How Do Existing Patients Book Appointments? As Each Appointment Is Recorded, Would Redundancy Not Occur?

Part B - Build a Human-Written Prototype:

Now, run both programs, and identify at least five limitations.

1. The Appointments Are Hard-Coded:
The Information within the statements are manually written into the source code itself. A receptionist cannot and should not enter in a new patient or appointment while the program is running or in general.

2. No Database Or Persistent Secondary Storage:
With the code provided the recorded appointments only exist while the program is running. Once the program closes, Python loses the information. This is a major limitation as within the case study SmartCare clearly requires reliable patient and appointment records.

3. No Appointment Booking Or Management Function. Code On Entails Manual Entry Of Appointments, And Management Is No Where In Sight.

4. There Is No Function Or Feature That Prevents Double Booking For Appointments.

5. Very Limited Patient Information, Patient Records And Appointment Records Are Not Connected To Patients.

6. No Unique Identifiers. Patient Names For Example Is Not Robust And May Cause Potential Issues Later On.

Initial Engineering Brief

Part C - Use AI as Tutor:

1. What the code does:

1. The appointment-booking function would be responsible for taking appointment information supplied by the user and storing it in the SmartCare database.

Conceptually, the process is:

Receptionist enters appointment details
↓
Python function
↓
Check the information
↓
Check practitioner availability
↓
MySQL database
↓
Appointment stored

For example, the function might receive:

- Patient ID
- Practitioner ID
- Appointment date
- Appointment time

It would then use SQL to check whether the practitioner already has an appointment at that date and time. If the time is available, the appointment can be inserted into the database.

This allows the system to move away from the original approach of manually writing appointments into Python variables.

Initial Engineering Brief

Part C - Use AI as Tutor:
2. Three limitations

1. Insufficient validation:

The function may accept invalid or incomplete information unless validation has been explicitly implemented.

For example, it could potentially receive an invalid date, an empty patient ID, or an invalid appointment time.

Why this matters: Incorrect information could be stored in the SmartCare database.

2. Duplicate bookings could still occur

Simply checking availability in Python may not be enough to guarantee that two users cannot book the same practitioner at the same time.

For example:

Receptionist A → checks 10:00 → available
Receptionist B → checks 10:00 → available
Receptionist A → books 10:00
Receptionist B → books 10:00

Both checks could occur before either appointment is inserted.

Why this matters: Duplicate bookings are one of the problems SmartCare specifically wants to solve.

A stronger design would use appropriate **database constraints/transactions** as an additional protection.

3. Limited error handling

If the MySQL database is unavailable or an SQL operation fails, the function needs to handle that failure appropriately.

Otherwise, the application could crash or leave the receptionist uncertain about whether the appointment was successfully booked.

A user-friendly system should distinguish between situations such as: Booking successful and Booking failed — please try again.

Initial Engineering Brief

Part C - Use AI as Tutor:

3. Suggested improvements:

Improvement	Purpose
Validate Input	Prevent invalid appointment information
Check Practitioner Availability	Reduce scheduling conflicts
Use Database Constraints	Provide stronger protection against duplicates
Add Error Handling	Prevent database failures from crashing the application
Use Appointment IDs	Give each appointment a unique identifier
Return A Clear Result	Tell the GUI whether booking succeeded or failed
Use Parameterised SQL	Make database queries safer and properly handle user input

The important software-engineering idea here is that the function should not just "put data into MySQL." It should enforce the rules that make an appointment valid.

For example:

Is the patient valid?
↓
Is the practitioner valid?
↓
Is the date/time valid?
↓
Is the practitioner available?
↓
Can the database safely create the appointment?
↓
SUCCESS

That is much closer to actual software engineering than simply writing an `INSERT` statement.

Initial Engineering Brief

Part C - Use AI as Tutor:

4. Why I wouldn't rewrite the whole application

At your current stage, it's better to improve **one piece of functionality at a time**.

For SmartCare, the appointment-booking function is a good component to study because it involves:

- user input
- validation
- business rules
- database queries
- error handling
- data integrity

Once you understand that component, the same concepts can be applied to cancellation, rescheduling, patient management and reporting.

5. Test your understanding

Question 1:
Why is checking whether a practitioner is available in Python alone not necessarily enough to completely prevent duplicate appointments?

Question 2:
If the receptionist enters an appointment with an invalid date, should the program allow Python to send it directly to MySQL? **Why or why not?**

Initial Engineering Brief

Part D - Generate an Alternative:

Prompt: Ignoring any previous context from both Memory and this conversation relating to my own code I would like you to use the SmartCare Case Study To create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Do not use or incorporate databases or GUIs.

Code Provided:

```python
def store_appointment(patient_name, practitioner_name, appointment_time):
    appointment = {
        "patient": patient_name,
        "practitioner": practitioner_name,
        "time": appointment_time
    }

    return appointment


# Example appointment
appointment1 = store_appointment(
    "Alice Smith",
    "Dr. John Doe",
    "10:00 AM"
)

print(appointment1)
```

Initial Engineering Brief

Part E - Compare Human and AI Versions:

Question	Human Version	Ai Version
Easy to understand?	No	Yes
Runs successfully?	Yes	Yes
Uses only required features?	No	Yes
Adds assumptions?	Yes	No
Handles errors?	Yes	No
Could I explain it?	Yes	Yes

SmartCare Requirements Engineering

Part A - Client Brief:

Already Completed Thoroughly In Previous Stage Refer To The Aforementioned.

Part B - Stakeholders and Scope:

Already Completed Thoroughly In Previous Stage Refer To The Aforementioned.

Part C - Functional Requirements:

Write 8-12 numbered functional requirements using FR-01, FR-02 and so on. Each should describe one observable capability.

- FR-01: Patient Record Creation
- FR-02: Patient Record Viewing
- FR-03: Patient Record Editing
- FR-04: Appointment Booking
- FR-05: Appointment Cancellation
- FR-06: Appointment Rescheduling
- FR-07: Practitioner Availability
- FR-08: Duplicate Appointment Prevention
- FR-09: Appointment Status
- FR-10: Appointment History Viewing
- FR-11: Operational Report Production

SmartCare Requirements Engineering

Part D - Non-Functional Requirements:

- FR-01: Reliability: The System Maintains Appointment Information Constantly And Does Not Lose A Successfully Recorded Appointment.

- FR-02: Data Integrity/Validity: The System Prevents Invalid, Incomplete Or Wrongly Formatted Appointment Information From Being Recorded. The System Also Maintains Consistent Relationships Within The Database Between Patients, Practitioners, And Appointments.

- FR-03: Usability/Accessibility: The System Has A More Complex Backend Yet This Allows The Frontend To Have A Friendly User Interface And Provides Accessibility.

- FR-04: Maintainability: Using The Concept Of Abstraction The System Is Ordered Into Multiple Different Parts. This Allows For Future Modifications Or Extensions Without Requiring Major Overhauls.

- FR-05: Testability: Due To The Previous Function, Almost All Functions Can Be Tested Independently.

- FR-06: Performance:

SmartCare Requirements Engineering

Part E - User Stories and Acceptance Criteria:

1. As A Receptionist, I Want The System To Prevent Me From Booking Unavailable Appointments, So That Practitioners Are Not Accidently Double Booked.

2. As A Member Of Management, I Want The System To Generate Basic Operational Reports Reliably, So That I Can Review Appointment Activity.

3. As A Practitioner, I Want To View A Patient's Relevant Appointment History, So That I Can Better Understand Their Previous Consultation(s) With The Clinic.

4. As A Patient, I Want The System To Allow Me To Book My Appointments Online, So That I Don't Have To Call Or Visit The Clinic.

SmartCare Requirements Engineering

Part F - AI Requirements Review:

Prompt: Act as a software requirements reviewer. Review the SmartCare requirements for ambiguity, inconsistency, missing clarification questions and testability. Do NOT invent new client requirements. For every suggestion, state whether it is based on evidence or is only a question/assumption requiring validation.

1. Overall Assessment:

The SmartCare case study is **clear about the problems and broad system scope**, but it is not yet detailed enough to serve as a complete implementation specification.

What is clearly established by evidence:

The case study explicitly establishes that:

- SmartCare is a small community clinic.

- It has multiple healthcare practitioners/GPs.

- Patient information and appointments are currently managed using spreadsheets, paper and manual processes.

- The system needs to support patients, practitioners and appointment management.

- The system should address the listed operational problems.

- The first version should remain simple and manageable rather than becoming a complex hospital information system.

SmartCare Requirements Engineering

Part F - AI Requirements Review:

1. Overall Assessment:

What Is Not Established:

The case study does not explicitly establish:

- Who will use the software;

- Whether patients themselves can access it;

- Whether practitioners can modify patient records;

- What information constitutes a patient record;

- What information constitutes an appointment;

- How practitioner availability is defined;

- Which appointment statuses are required;

- How long appointment history must be retained;

- What operational reports are required;

- What authentication/access-control rules are required;

- Whether the system is intended to be online;

- Whether appointments can be booked directly by patients.

Those should therefore be treated as clarification questions rather than requirements.

SmartCare Requirements Engineering

2. Inconsistencies or potential scope problems:

There aren't many **actual contradictions** in the case study.

However, there is one important potential scope issue.

"Simple system" vs. potentially broad functionality

The case study says:
"The organisation does not want a complex hospital information system."

While also asking for management of:
- Patients;
- Practitioners;
- Appointments;
- Appointment history;
- Reporting;
- Availability.

These aren't contradictory, but the scope could expand rapidly if every area is interpreted broadly.

Evidence: Both the broad management requirement and the explicit scope limitation are stated.

Question requiring validation:
What functionality is considered essential for the first version, and what functionality should explicitly be excluded from the first version?

This would help prevent scope creep.

SmartCare Requirements Engineering

Part G - VERIFY the AI Review:

Classify each significant AI suggestion as Accepted, Modified, Rejected, or Unverified. Explain the evidence used.

1. Overall Assessment: Accepted. The Information provided by the response was all derived from evidence which could be found in the case study.

2. Inconsistencies or potential scope problems: Accepted. The Information provided by the response was all derived from evidence which could be found in the case study.

Reflection:

In 150-250 words: What did AI notice that you missed? What did AI invent or overreach on? Which requirement changed after review? Why must requirements have evidence?

Overall the generative Ai model I was using did not notice anything that I missed or had not already mentioned or had in mind. The response it provided me was actually mostly built from my own user inputs and insights. However what the Generative Artificial Intelligence Model did assist in was clarification and elaboration on my previous insights.

The AI model did not invent anything, however what the model did do was misinterpret the Case Study In Point 2: Inconsistencies or potential scope problems; Question requiring validation. The question the response asked has merit however it is clearly stated what functionality the first version should have despite the case study being ambiguous on the details of the feature.

Requirements must have evidence otherwise they would be considered assumptions, and assumptions may not be what the client requires. Furthermore this may add additional issues to the system itself.

From Problems to Requirements

Activity 1 - Stakeholder Map:

Stakeholder	Need	Potential Conflict
Patient	- View Availability Of Practitioners - More Robust Appointment Booking System - Appointment Management System	Organisation Management Patients Want A Robust Appointment Management System Which May Conflict With The Organisations And Management's Desire For A "Simple" System. Clarification On Simple Will Assist In Solving This Issue.
Organisation	- Simple Software System: Not Complex Like Hospital Information Systems	Patient
Management	- Operational Report Production - Duplicate Appointment Booking Prevention - Simple Software System	Patient
Practitioner	- Reliable Appointment History - Reliable Patient Record Location	

From Problems to Requirements

Activity 1 - Stakeholder Map:

There are no major conflicts, I have identified between the aforementioned stakeholder requirements. The proposed software architecture will separate the systems logical components through the concept of abstraction. This would mean each system will have its own responsibilities and logic, while the only thing they are sharing is data from a Database. The systems themselves may not inherently conflict yet they are still dependent on one another, since they most likely will access or update shared information.

Activity 2 - Functional or Non-Functional?

1. The system shall allow staff to cancel an appointment. **Functional**.

2. The system should remain responsive for the course-scale dataset. **Non Functional.**

3. Core business logic should be independently testable. **Non Functional.**

4. The system shall search for a patient by ID. **Functional.**

From Problems to Requirements

Activity 3 - Repair Ambiguous Requirements:

1. The System Should Be Easy To Use.

Problem: This Statement Does Not Tell Us Who The System Should Be Easy To Use For. The Patients? Management? Or The Practitioners? If So The Term “Easy” Could Mean Many Many Different Things.

Clarification Question: Could You Elaborate On The Statement “Easy To Use”? Who Should This System Be Designed For To Use? What Features Does “Easy To Use” Entail?

2. Patient Search Should Be Fast.

Problem: What Does The Term Fast Entail? Fast Could Mean Very Different Features Ranging From: Fast Return Of A Searched Query To A Filter Option That Allows For Fast Location Of Specific Types Of Patients/Patient Appointments.

Clarification Question: Could You Elaborate On The Term “Fast”? Who Should This System Be Designed For, Practitioners, Management Or The Organization Itself? What Composite Of Features Would A “Fast Search” Entail?

3. The System Should Securely Manage Data.

Problem: What Does Statement “Securely Manage” Entail? Do They Want A Secure Data System? Or Are They After A Secure Data System That Also Manages Said Data? What Kind Of Data Are They Managing, This Would Change The Definition Of Safety. The Safety Of Patient Data Would Differ From The Safety Of Operational Clinic Data For Example.

Clarification Question: Could You Elaborate On The Statement "Securely Manage", Are You Looking For A System That Provides Both Security And Management Of Data? What And Whose Data Is This System Supposed To Securely Manage?

From Problems to Requirements

4. Appointments should normally be easy to cancel.

Problem: What Does An Easily Cancelable Appointment Entail? This Could Mean A Range Of Things From Accessibility, User Interface On A Website, To A System Which Notifies Receptionists When An Appointment Needs To Be Cancelled.

Clarification Question: Could You Elaborate On The Statement “Easily Cancelable Appointment”? Where And How Would You Like The Appointments To Be Cancelled In The First Place?

From Problems to Requirements

Activity 4 - AI Requirements Audit:

Classify each suggestion: Confirmed / Assumption requiring validation / Unsupported / Out of scope.

Ai Suggestion	Classification	Evidence/Reason
Patients receive SMS reminders.	Assumption Requiring Validation	“manual cancellation processes;”
Facial recognition login.	Out Of Scope	A simple software system that can initially support patient, practitioner and appointment management.
Receptionists create appointments.	Assumption Requiring Validation	“manual cancellation processes;”
Online payment.	Unsupported	“manual cancellation processes;”
Practitioners view schedules.	Confirmed	“limited visibility of practitioner availability;”
AI recommends treatments.	Out Of Scope	“The organisation does not want a complex hospital information system.”
Cancelled appointments remain in history.	Assumption Requiring Validation	“difficulty producing basic operational reports.”

SmartCare Domain Modelling

Requirement-to-Concept Trace:

Requirement	Concept	State/Behaviour	Decision
Duplicate Appointment Booking Prevention	Appointment	Checks Whether An Appointment Already Exists For The Selected Practitioner/ Time	Allows Or Reject Booking
Reliable Way To Locate Patient Records	Patient Record	Stores And Retrieves Patient Records	Returns The Correct Patient Record
Clear Visibility Of Practitioner Availability	Practitioner/ Availability	Stores And Displays Available Appointment Openings/ Times	Show Whether A Practitioner Or Timeslot Is Available
Non-Manual Appointment Cancellation Process	Appointment	Alters An Appointments Status To Cancelled	Marks The Appointment As Cancelled
Reliable Method To Obtain Appointment History	Appointment History	Stores And Retrieves Previous Appointment History	Returns The Correct/Relevant Appointment History
Simple Software System	Software System	Provide Only The Required Functionality	Keeps The System Within The Required Scope
Simplicity In Basic Operational Report Production	Operational Report	Generates Operational Reports From The Clinics Data	Produces The Requested Report Correctly

SmartCare Domain Modelling

B - Candidate Classes:

Record candidate concepts, supporting requirements, state and behaviour.

Candidate Class	Supporting Requirement	State	Behaviour
Patient	Reliable Method In Locating Patient Records	Patient ID, Full Name, Contact Details	Create, Find, View, And Edit Patient Records/ Personal Information
Practitioner	Clear Visibility Of Practitioner Availability/ Timeslots	Practitioner ID, Full Name, Availability, Specality	Create, View And Edit Availability
Appointment	1. Duplicate Booking Prevention 2. Cancellation 3. Management 4. Appointment History	Appointment ID, Patient ID, Practitioner ID, Date/Time, Status	Book, Cancel, Reschedule And View Status Of Appointments
Appointment History	Reliable Method In Obtaining Patient Appointment History	Previous Appointment	Retrieves And Displays Appointment History
Operational Report	Simplicity In Basic Operational Report Production	Report Data/Details (Unknown)	Generates Basic Operational Reports

SmartCare Domain Modelling

C - CRC Cards:

Create CRC cards for Patient, Practitioner and Appointment.

Class	Responsibilities	Collaborators
Patient	- Store Patient Information - Create Patient Record - Locate Patient Record - Update Patient Record - View Appointment History	Appointment
Practitioners	- Store Practitioner Data - Maintain Practitioner Availability - Provide Available Appointment Times - View Scheduled Appointments	Appointment
Appointment	- Store Appointment Information - Check Practitioner Availability - Prevention Of Duplicate Bookings - Appointment Cancellation - Appointment Management/ Rescheduling - Maintenance Of Appointment Status/ History	Patient AND Practitioner

SmartCare Domain Modelling

D - UML Model:

Draw classes, attributes, operations, associations and multiplicities.

SmartCare Domain Modelling

E - AI Design Review:

Ask AI to suggest classes and relationships using only confirmed requirements; require supporting requirement IDs.

Candidate Class	Supporting Requirement	State	Behaviour
Patient	Reliable way to locate patient records	Patient ID, name, contact details	Create, find, view and update patient records
Practitioner	Clear visibility of practitioner availability	Practitioner ID, name, availability	View and update availability
Appointment	Duplicate booking prevention; cancellation; appointment history	Appointment ID, patient, practitioner, date/time, status	Book, cancel, reschedule and check availability
Appointment History	Reliable method to obtain appointment history	Previous appointments	Retrieve and display appointment history
Operational Report	Simplicity in basic operational report production	Report data/details	Retrieve and display appointment history

SmartCare Domain Modelling

F - Compare and Decide:

Record at least one accepted, modified and rejected AI suggestion.

No major differences besides wording, and AI's lack of technical terms as The Generative AI used context, and personal insight from previous prompts which I had inputted due to previous AI related activities. Furthermore my design is less ambiguous.